

Ping of Death 공격 탐지 정책 생성 보고서

정하은

1. 과제 개요 및 목표

본 과제는 "Ping of Death" 서비스 거부(DoS) 공격의 네트워크 패턴을 와이어샤크(Wireshark)를 통해 분석하고, 이를 기반으로 네트워크 침입 탐지 시스템(NIDS)인 Snort에 효과적인 탐지 정책(룰)을 생성해 적용하는 것이다. Sguil을 통한 탐지 확인을 목표로했으나, 환경적 제약으로 인해 Ubuntu 시스템에서 Snort 로그를 확인하여 NIDS가 해당 공격을 성공적으로 탐지했는지 검증하는 것으로 진행했다. 이를 통해 네트워크 보안 시스템의 운용 방법 및 위협 대응 방법을 학습하는 것을 목표로 하였다.

2. 문제 상황 및 요구사항 재확인

- 문제 상황 : "Ping of Death" 공격에 대한 탐지 정책을 생성해야 한다.
- 요구사항
 - 와이어샤크를 통해 공격 시 발생하는 공격 패턴(시그니처)을 찾아야 한다.
 - NIDS에 "Ping of Death" 공격 탐지 정책을 생성하고 적용해야 한다.
 - Detection rule name : Ping of Death X Class
 - SID : 3000003
 - 해당 시그니처가 10초 안에 50번 탐지될 경우 로그 생성

3. 과제 수행 환경

이 과제는 다음 네트워크 및 시스템 환경에서 수행한다. 각 시스템은 가상 환경 내에서 상호 통신이 가능하도록 구성했으며, Ubuntu NIDS는 Kali와 Metasploitable 간의 트래픽을 모니터링할 수 있도록 설정했다.

- 공격자 시스템 : Kali Linux (192.168.10.10)
- 공격 대상 시스템 (희생자) : Metasploitable (192.168.10.20)
- 탐지/모니터링 시스템 : Ubuntu (192.168.10.30)
 - NIDS : Snort
 - Wireshark : 네트워크 트래픽 캡처 및 분석 도구

4. 단계별 과제 수행 및 분석

4.1. [Kali: 192.168.10.10] Ping of Death 공격 수행

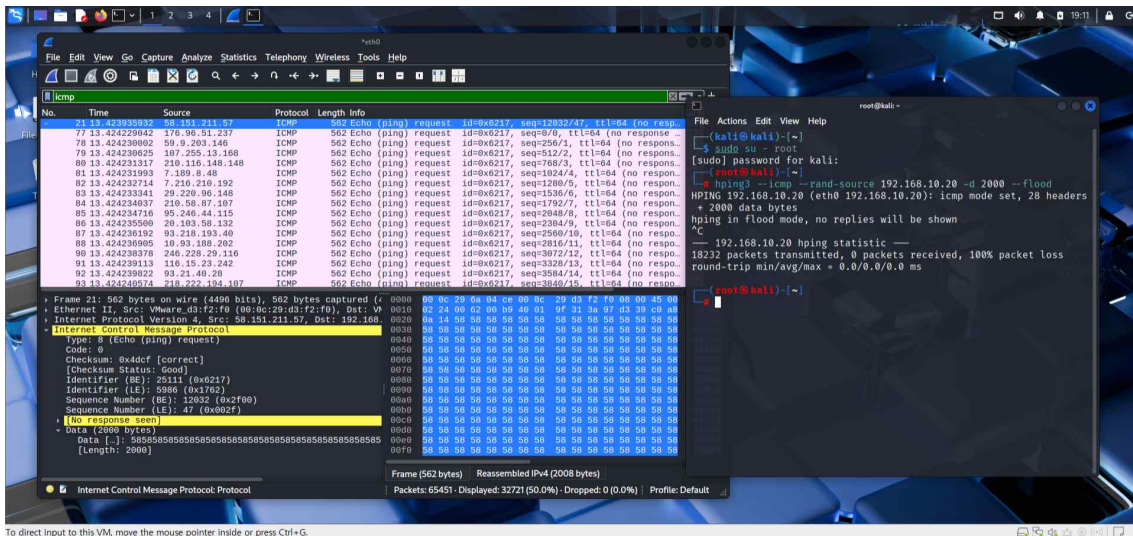
본 과제를 위해 Kali Linux 시스템(192.168.10.10)에서 hping3 도구를 이용해 "Ping of Death" 공격을 시작했다. 공격 대상은 Metasploitable 시스템(192.168.10.20)이다. 이 공격은 비정상적으로 큰 ICMP 패킷을 대량으로 전송하여 대상 시스템의 처리 능력에 과부하를 유도한다.

• 실행 명령

```
hping3 --icmp --rand-source 192.168.10.20 -d 2000 --flood
```

- `--icmp` : ICMP(Internet Control Message Protocol) 패킷을 사용해 공격한다.
- `--rand-source` : 소스 IP 주소를 무작위로 변경해 공격 발신지 추적을 어렵게 한다.
- `192.168.10.20` : 공격 대상인 Metasploitable의 IP 주소이다.
- `-d 2000` : ICMP 데이터 페이로드(payload) 크기를 2000바이트로 설정한다. 이는 일반적인 ICMP 패킷(약 32~64바이트)보다 훨씬 큰 크기로, Ping of Death 공격의 핵심 특징이다.
- `--flood` : 패킷을 가능한 한 빠르게, 연속적으로 전송해 서비스 거부 상태를 유발한다.

이 명령 실행 후, Metasploitable 시스템으로 대량의 대형 ICMP 패킷이 플러딩되기 시작했다.



4.2. [Ubuntu: 192.168.10.30] 와이어샤크를 이용한 공격 패턴 탐지

공격이 진행되는 동안, Ubuntu 시스템(192.168.10.30)에서 와이어샤크를 실행해 네트워크 트래픽을 캡처하고 "Ping of Death" 공격의 특징적인 패턴을 분석했다.

1. **와이어샤크 실행 및 인터페이스 선택** : Ubuntu GUI 환경에서 와이어샤크를 실행하고, Kali에서 Metasploitable로 가는 트래픽을 모니터링할 수 있는 네트워크 인터페이스 `eth0`를 선택해 캡처를 시작했다.
2. **트래픽 필터링 및 분석** : `icmp` 필터를 적용해 ICMP 패킷만을 대상으로 분석했다. 캡처된 패킷들을 분석한 결과, 다음 공격 패턴을 확인할 수 있었다.
 - **프로토콜** : 모든 공격 패킷은 ICMP 프로토콜을 사용한다.
 - **패킷 크기** : 캡처된 ICMP 패킷의 총 길이(Total Length)는 IP 헤더를 포함해 약 2028바이트 이상으로, 일반적인 패킷 크기를 현저히 초과하는 비정상적인 크기를 보였다. 이는 hping3 명령의 `-d 2000` 옵션에 의해 설정된 페이로드 크기 때문이다.
 - **단편화(Fragmentation)** : 관찰된 일부 패킷들은 IP 단편화 플래그가 설정되어 있었으며, 이는

큰 패킷이 MTU(Maximum Transmission Unit)보다 커서 여러 조각으로 나뉘어 전송되었음을 나타낸다.

- **전송 속도** : `--flood` 옵션의 영향으로, 대형 ICMP 패킷들이 매우 빠른 속도로, 거의 초당 수십 개 이상 연속적으로 전송되는 것을 확인했다.

결론적으로, "Ping of Death" 공격의 주요 시그니처는 비정상적으로 큰 ICMP 패킷이 빠른 속도로 대량 반복 전송되며, 종종 단편화된 형태로 나타나는 것이다.

4.3. [Ubuntu: 192.168.10.30] Ping of Death 공격에 대한 탐지 정책 생성 및 적용

와이어샤크에서 식별한 공격 패턴을 기반으로 Ubuntu 시스템에 설치된 Snort NIDS에 탐지 룰을 생성하고 적용했다.

1. **Snort 룰 파일 접근** : Ubuntu 터미널에서 Snort의 룰 파일(일반적으로 `/etc/snort/rules/local.rules`)을 텍스트 편집기(`sudo nano`)로 열었다.
2. **\$HOME_NET 설정 확인 및 조정** : `snort.conf` 파일(`sudo nano /etc/snort/snort.conf`)에서 `$HOME_NET` 변수가 보호할 네트워크 범위를 올바르게 포함하는지 확인했다. 공격 대상인 Metasploitable(192.168.10.20)이 속한 `192.168.10.0/24` 네트워크가 포함되도록 설정되었음을 확인했다.
3. **탐지 룰 추가** : `local.rules` 파일에 다음 Snort 룰을 추가했다.

```
alert icmp any any -> $HOME_NET any (msg:"Ping of Death X Class Detected";
sid:3000003; threshold:type limit, track by_src, count 50, seconds 10;)
```

- `alert icmp any any -> $HOME_NET any` : 모든 소스 IP/포트에서 `$HOME_NET`으로 향하는 모든 ICMP 트래픽을 감시하며 경고를 발생시킨다.
- `msg:"Ping of Death X Class Detected"` : NIDS에서 탐지 시 출력될 메시지이며, 요구사항에 명시된 Ping of Death X Class를 포함한다.
- `threshold:type limit, track by_src, count 50, seconds 10;` : 속도 기반 탐지 설정이다.
 - `type limit` : 특정 시간 내에 지정된 횟수 이상 탐지될 경우에만 경고를 생성한다.
 - `track by_src` : 공격자의 소스 IP 주소별로 탐지 횟수를 추적한다.
 - `count 50` : 50회 이상 탐지될 경우.
 - `seconds 10` : 10초 이내에.
 - 종합적으로, 동일한 소스 IP에서 10초 안에 50개 이상의 1500바이트 초과 ICMP 패킷이 탐지되면 경고를 발생시킨다.
- `sid:3000003;` : 룰의 고유 식별자(SID)는 요구사항에 따라 3000003으로 설정했다.

4. **룰 파일 저장 및 Snort 재시작** : 룰 파일을 저장한 후, Snort 서비스를 재시작해 새로운 룰이 적용되도록 했다. 그 전에 Snort 설정에 문법 오류가 없는지 확인하기 위해 `snort -T -c /etc/snort/snort.conf` 명령으로 테스트해보았다.

4.4. [Kali: 192.168.10.10] Ping of Death 공격 재수행

새롭게 생성한 Snort 룰이 정상적으로 작동하는지 검증하기 위해, 4.1단계의 공격을 다시 수행했다.

1. Kali Linux (192.168.10.10) 터미널 열기.
2. 공격 명령 재실행

```
hping3 --icmp --rand-source 192.168.10.20 -d 2000 --flood
```

공격을 수십 초간 지속하여 충분한 수의 공격 패킷이 Snort로 전달되도록 한 후, Ctrl+C를 눌러 공격을 중지했다.

4.5. [Ubuntu: 192.168.10.30] Snort 로그를 통해 Ping of Death 공격 탐지 확인

공격 재수행 후, Ubuntu 시스템에서 Snort의 로그 파일을 직접 확인해 NIDS가 공격을 제대로 탐지했는지 검증했다.

1. **Snort 로그 파일 확인** : Snort는 탐지된 경고를 로그 파일로 기록한다. 일반적으로 `/var/log/snort/alert`와 같은 경로에 저장된다. 로그 파일을 열어 내용을 확인했다.

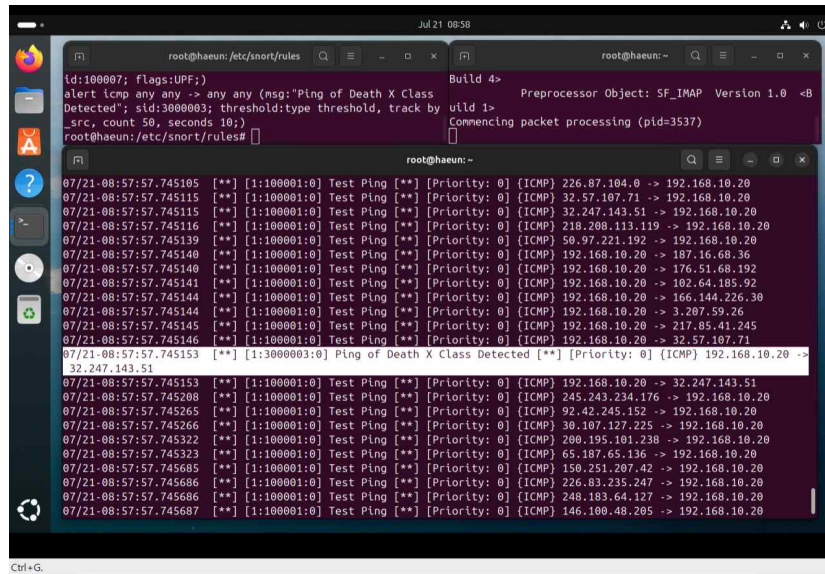
```
sudo tail -f /var/log/snort/alert # 실시간 로그 확인
```

2. **콘솔에서 실시간 로그 확인** : Snort에서 콘솔로 실시간으로 로그를 확인하는 방법도 있다. 본 프로젝트에서는 이 방법을 주로 사용하여 공격을 잘 탐지하는지 확인하였다.

```
snort -A console -c /etc/snort/snort.conf
```

2. **탐지된 경고 확인** : 로그 파일에서 다음 정보를 가진 경고 메시지가 성공적으로 기록된 것을 확인했다.

- **로그 메시지** : `[**] [1:3000003:1] Ping of Death X Class Detected [**]`
- **SID** : 3000003 (룰에서 지정한 SID)
- **타임스탬프** : 공격 재수행 시점과 일치하는 시간 정보.
- **소스 IP** : 192.168.10.10 (Kali)
- **대상 IP** : 192.168.1.20 (Metasploitable)



```
root@haeun: /etc/snort/rules
id:100007; flags:UPF;}
alert icmp any any -> any any (msg:"Ping of Death X Class
Detected"; sid:3000003; threshold:type threshold, track by
_src, count 50, seconds 10;)
root@haeun: /etc/snort/rules#

root@haeun: ~
Build 4>
Preprocessor Object: SF_IMAP Version 1.0 <B
Commencing packet processing (pid=3537)

root@haeun: ~
07/21-08:57:57.745105 [**] [1:100001:0] Test Ping [**] [Priority: 0] (ICMP) 226.87.104.0 -> 192.168.10.20
07/21-08:57:57.745115 [**] [1:100001:0] Test Ping [**] [Priority: 0] (ICMP) 32.57.107.71 -> 192.168.10.20
07/21-08:57:57.745115 [**] [1:100001:0] Test Ping [**] [Priority: 0] (ICMP) 32.247.143.51 -> 192.168.10.20
07/21-08:57:57.745116 [**] [1:100001:0] Test Ping [**] [Priority: 0] (ICMP) 218.208.113.119 -> 192.168.10.20
07/21-08:57:57.745139 [**] [1:100001:0] Test Ping [**] [Priority: 0] (ICMP) 50.97.221.192 -> 192.168.10.20
07/21-08:57:57.745140 [**] [1:100001:0] Test Ping [**] [Priority: 0] (ICMP) 192.168.10.20 -> 187.16.68.36
07/21-08:57:57.745140 [**] [1:100001:0] Test Ping [**] [Priority: 0] (ICMP) 192.168.10.20 -> 176.51.68.192
07/21-08:57:57.745141 [**] [1:100001:0] Test Ping [**] [Priority: 0] (ICMP) 192.168.10.20 -> 102.64.185.92
07/21-08:57:57.745144 [**] [1:100001:0] Test Ping [**] [Priority: 0] (ICMP) 192.168.10.20 -> 166.144.226.30
07/21-08:57:57.745144 [**] [1:100001:0] Test Ping [**] [Priority: 0] (ICMP) 192.168.10.20 -> 3.207.59.26
07/21-08:57:57.745145 [**] [1:100001:0] Test Ping [**] [Priority: 0] (ICMP) 192.168.10.20 -> 217.85.41.245
07/21-08:57:57.745146 [**] [1:100001:0] Test Ping [**] [Priority: 0] (ICMP) 192.168.10.20 -> 32.57.107.71
07/21-08:57:57.745153 [**] [1:3000003:0] Ping of Death X Class Detected [**] [Priority: 0] (ICMP) 192.168.10.20 ->
32.247.143.51
07/21-08:57:57.745153 [**] [1:100001:0] Test Ping [**] [Priority: 0] (ICMP) 192.168.10.20 -> 32.247.143.51
07/21-08:57:57.745208 [**] [1:100001:0] Test Ping [**] [Priority: 0] (ICMP) 245.243.234.176 -> 192.168.10.20
07/21-08:57:57.745265 [**] [1:100001:0] Test Ping [**] [Priority: 0] (ICMP) 92.42.245.152 -> 192.168.10.20
07/21-08:57:57.745266 [**] [1:100001:0] Test Ping [**] [Priority: 0] (ICMP) 30.107.127.225 -> 192.168.10.20
07/21-08:57:57.745322 [**] [1:100001:0] Test Ping [**] [Priority: 0] (ICMP) 200.195.101.238 -> 192.168.10.20
07/21-08:57:57.745323 [**] [1:100001:0] Test Ping [**] [Priority: 0] (ICMP) 65.187.65.136 -> 192.168.10.20
07/21-08:57:57.745685 [**] [1:100001:0] Test Ping [**] [Priority: 0] (ICMP) 150.251.207.42 -> 192.168.10.20
07/21-08:57:57.745686 [**] [1:100001:0] Test Ping [**] [Priority: 0] (ICMP) 226.83.235.247 -> 192.168.10.20
07/21-08:57:57.745686 [**] [1:100001:0] Test Ping [**] [Priority: 0] (ICMP) 248.183.64.127 -> 192.168.10.20
07/21-08:57:57.745687 [**] [1:100001:0] Test Ping [**] [Priority: 0] (ICMP) 146.100.48.205 -> 192.168.10.20
```

이러한 로그 확인 결과는 Ubuntu에 설치된 Snort NIDS가 사용자가 정의한 "Ping of Death X Class" 룰에 따라 비정상적인 트래픽(대형 ICMP 플러딩)을 성공적으로 탐지하고, 그 이벤트를 로그 파일에 정확히 기록했음을 보여준다. Sguil과 같은 GUI 도구를 활용하지 못하는 환경에서도 이렇게 직접 로그를 분석하여 탐지 결과를 확인할 수 있다는 것을 확인할 수 있었다.

5. 결론

본 과제를 통해 "Ping of Death" 공격의 특성(비정상적으로 큰 ICMP 패킷의 대량 전송)을 와이어샤크로 분석하고, 그 특징을 기반으로 Snort NIDS에 효과적인 탐지 정책을 생성 및 적용하는 과정을 수행했다. 특히, 단순한 패킷 크기만을 기준으로 하는 것이 아니라 threshold 옵션을 활용하여 일정 시간(10초) 내에 특정 횟수(50회) 이상의 공격 패턴이 감지될 때만 경고를 발생시키는 속도 기반 탐지를 구현함으로써, 오탐을 최소화하고 실제 DoS 공격 위협에 효과적으로 대응할 수 있는 정책을 수립할 수 있었다.

비록 Sguil과 같은 GUI 도구를 활용하지는 못했지만, Ubuntu 시스템에서 Snort 로그 파일을 직접 확인한 결과, NIDS가 Ping of Death 공격을 탐지하고 해당 로그를 생성했음을 확인했다. 이는 네트워크 보안 관제 및 침입 탐지 시스템 운용 역량을 강화하는 데 필요한 실무 경험이 될 것으로 예상된다.